GCC-Wireless KiCad workspace. Open a sheet: double-click it (or select it, Ctrl+Tab).
PhobGCC 2.0.5 is an unmodified copy of upstream (see phob/SOURCE.md). The other sheets are ours.
Guide: docs/HARDWARE.md

PhobGCC 2.0.5

File: phob/PhobGCC_2_0_0_proto_1.kicad_sch

Ref nice!nano v2 pinout

File: ref_nicenano.kicad_sch

Ref GameCube port pinout

File: ref_gc_port.kicad_sch

Ref RP2040 bridge pins

File: ref_rp2040_bridge.kicad_sch

Controller add-on TX

File: tx_addon.kicad_sch

Console receiver RX

File: rx_receiver.kicad_sch

GCC-Wireless (GPL-3.0); Phob sheet CERN-OHL-S v2

Sheet: /
File: GCC-Wireless.kicad_sch

Title: GCC-Wireless: schematics for the wireless PhobGCC

Size: A3	Date: 2026-11-05	**Rev:**
KiCad E.D.A. 10.0.7		Id: 1/7

Crystal

Power

Flash

Make sure C10 is close to pin 44 of RP2040

Make sure C8 is close to pin 45 of RP2040

Make sure R3 and R4 are close to RP2040

Raspberry Pi
RP2040

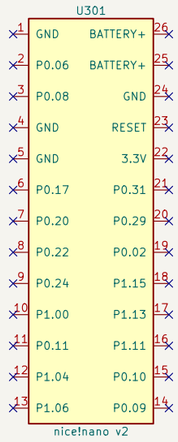
RP2040

Sheet: /PhobGCC 2.0.5/
File: PhobGCC_2_0_0_proto_1.kicad_sch

Title:

Size: A3 | Date: | **Rev:**
KiCad E.D.A. 10.0.7 | Id: 2/7

Pin names copied from the official v2 pinout image
 https://nicekeyboards.com/docs/nice-nano/pinout-schematic
  fetched 2026-11-05 (file pinout-v2.png on that page).
Pin numbers 1–26 are ours (the image has none): left column 1–13 top to bottom
(USB-C end at the top), right column 14–26 bottom to top (BATTERY+ top = 26, 25).
Arduino labels on the image: D1=P0.06 D0=P0.08 D2=P0.17 D3=P0.20 D4=P0.22 D5=P0.24
D6=P1.00 D7=P0.11 D8=P1.04 D9=P1.06 | D21=P0.31 D20=P0.29 D19=P0.02 D18=P1.15
D15=P1.13 D14=P1.11 D16=P0.10 D10=P0.09.
Image legend: when P0.13 is set low, power to the 3.3V pin is cut off (external VCC cutoff).
Image note: bridging the BOOST jumper raises the charge rate from 100 mA to 500 mA.



Reference only – no wiring. X marks = not connected on purpose.

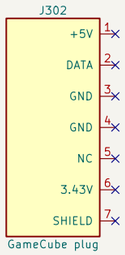
GCC-Wireless (GPL-3.0)

Sheet: /Ref nice!nano v2 pinout/
File: ref_nicenano.kicad_sch

Title: Ref nice!nano v2 pinout

Size: A3	Date: 2026-11-05	**Rev:**
KiCad E.D.A. 10.0.7		Id: 3/7

Source: https://www.int03.co.uk/crema/hardware/gamecube/gc-control.html (read 2026-11-05).
Pin numbers are that page's own, viewed looking into the console socket (so a plug is the mirror).
1 yellow: 5 V supply (used by the rumble motor)
2 red: DATA, bidirectional, pull-up to 3.43 V
3 green: ground 4 white: ground (Skillz adapter ties 3 and 4 together)
5: not connected by the official controller
6 blue: 3.43 V logic supply (page measured 3.43 V on a PAL console; calls 3.3 V a likely fine choice)
7 black: cable shield / ground
The page's own caveat: pinout is deduced from multimeter and oscilloscope work, no guarantee.
Cross-check: Phob J1 (cable pads) in the Phob sheet = 1 GCC_3.3V, 2 +5V, 3 GCC_DATA, 4-6 GND
(docs/facts/phobgcc.md, netlist of PhobGCCv2-HW ff645aa). Same four signals, different numbering.
Bridge data pin: GP27 (upstream/gcwireless_bridge @ 5c31147, src/main.cpp line 10).



Reference only — no wiring.

GCC-Wireless (GPL-3.0)

Sheet: /Ref GameCube port pinout/
File: ref_gc_port.kicad_sch

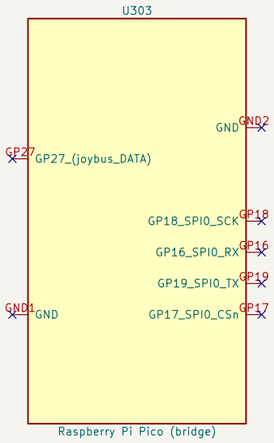
Title: Ref GameCube port pinout

Size: A3	Date: 2026-11-05	**Rev:**
KiCad E.D.A. 10.0.7		Id: 4/7

Source: upstream/gcwireless_bridge @ 5c31147 (https://github.com/heatherspacek/gcwireless_bridge).
Board: CMakeLists.txt sets no board, so the Pico SDK default (pico) applies; build-support/build-bridge.sh passes -DPICO_BOARD=pico. The SDK header boards/pico.h (SDK 2.1.1) defines the default SPI pins.

GPIO	use	source line
GP27	joybus DATA to console	src/main.cpp:10 _pinTX = 27 ; enterMode(_pinTX, ...)
GP16	SPI0 RX (data in)	src/main.cpp:87 PICO_DEFAULT_SPI_RX_PIN (pico.h: 16)
GP18	SPI0 SCK (clock in)	src/main.cpp:88 PICO_DEFAULT_SPI_SCK_PIN (pico.h: 18)
GP19	SPI0 TX (data out)	src/main.cpp:89 PICO_DEFAULT_SPI_TX_PIN (pico.h: 19)
GP17	SPI0 CSn (chip select)	src/main.cpp:90 PICO_DEFAULT_SPI_CSN_PIN (pico.h: 17)
GP25	onboard LED, set high	src/joybus.cpp:100 gpio_put(25, 1)

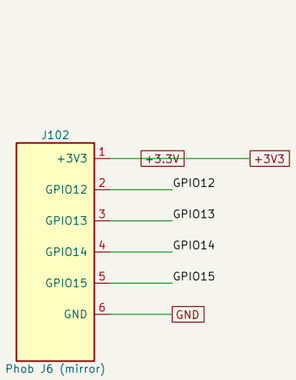
SPI runs as slave (src/main.cpp:86 spi_set_slave), 1 MHz setting, 10-byte reads (BUF_LEN = 10).
Pin numbers on the symbol are GPIO numbers; physical pin numbers were not checked.

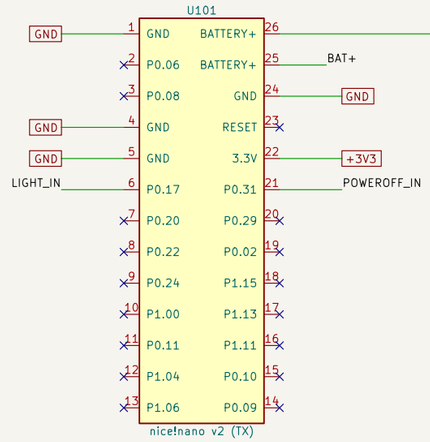


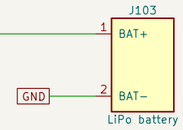
Reference only — no wiring.
GCC-Wireless (GPL-3.0)

Sheet: /Ref RP2040 bridge pins/ File: ref_rp2040_bridge.kicad_sch		
Title: Ref RP2040 bridge pins		
Size: A3	Date: 2026-11-05	**Rev:**
KiCad E.D.A. 10.0.7		Id: 5/7

TBD (not invented): which nice!nano pins receive Phob GPIO12–GPIO15 and whether the link is SPI or UART.
Upstream gcwireless TX code (src/tx.cpp @ 576878d) has no Phob link at all: it only reads P0.17 (light sensor test, INPUT_PULLUP, tx.cpp lines 7 and 96) and P0.31 (power-down input, tx.cpp lines 5 and 88).
Those two are labelled LIGHT_IN and POWEROFF_IN here; they are firmware test pins, not final wiring.
TBD: power direction on J6 pin 1. The Phob 3.3 V rail is normally fed by +5V -> AMS1117 -> D1 (docs/facts/phobgcc.md); feeding +3V3 from the nice!nano 3.3V pin is unverified (docs/facts/power.md).
TBD: battery connector type (nice!nano has BATTERY+ pads; pinout page above).
BAT+ goes to both BATTERY+ pins, as the official image shows two.







BAT+

Drawing only. TBD items are decisions the upstream code does not make.

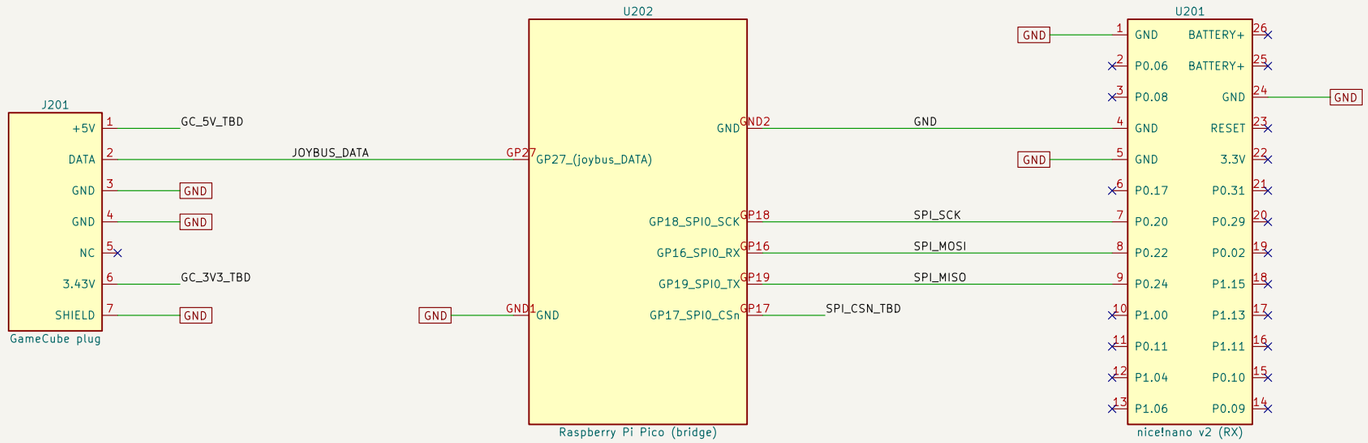
GCC–Wireless (GPL–3.0)

Sheet: /Controller add–on TX/
File: tx_addon.kicad_sch

Title: Controller add–on TX

Size: A3	Date: 2026–11–05	**Rev:**
KiCad E.D.A. 10.0.7		Id: 6/7

SPI (Serial Peripheral Interface) nets, from the code:
  nRF52 side: upstream/gcwireless src/rx.cpp @ 576878d lines 72–76: nrf_spim_pins_set(NRF_SPIM3,
    PIN_020 /*SCK*/, PIN_022 /*MOSI*/, PIN_024 /*MISO*/). With build-support/gcwireless/pins.h (ours),
    PIN_020 = P0.20, PIN_022 = P0.22, PIN_024 = P0.24 (D3, D4, D5 on the official image).
  RP2040 side: upstream/gcwireless_bridge @ 5c31147 src/main.cpp lines 85–90 (slave on the SDK default
    SPI0 pins: SCK GP18, RX GP16, TX GP19, CSn GP17; values from pico.h in Pico SDK 2.1.1).
  So SCK P0.20 -> GP18, MOSI P0.22 -> GP16 (the Pico's RX), MISO P0.24 <- GP19 (the Pico's TX).
  Direction wording (MOSI/MISO) is our reading of which side is master; the code does not say it outright.
Joybus DATA: bridge GP27 (src/main.cpp:10) to GameCube plug pin 2 (int03 numbering).
TBD: chip select. rx.cpp lines 66–71 has the nRF CSN setup (PIN_002) commented out, so no nRF pin is chosen;
  GP17 (CSn) is drawn dangling as SPI_CSN_TBD.
TBD: how the receiver is powered (GC +5V and 3.43V are labelled only); the code is silent.
Note the bridge currently copies only byte 0 of the SPI data (main.cpp line 104), and reads 10 bytes.



Drawing only. TBD items are decisions the upstream code does not make.

GCC-Wireless (GPL-3.0)

Sheet: /Console receiver RX/
File: rx_receiver.kicad_sch

Title: Console receiver RX

Size: A3	Date: 2026-11-05	**Rev:**
KiCad E.D.A. 10.0.7		Id: 7/7